In [1]:
```
# All Libraries required for this lab are listed below. The libraries pre-installed on Skills Network Labs are commented.
%pip install -qy pandas==1.3.4 numpy==1.21.4 seaborn==0.9.0 matplotlib==3.5.0 folium
# Note: If your environment doesn't support "%pip install", use "!mamba install"
```

```
Note: you may need to restart the kernel to use updated packages.
[optparse.groups]Usage:[/]
  D:\python\python.exe -m pip install \[options] <requirement specifier> \[package-index-options] ...
  D:\python\python.exe -m pip install \[options] -r <requirements file> \[package-index-options] ...
  D:\python\python.exe -m pip install \[options] [-e] <vcs project url> ...
  D:\python\python.exe -m pip install \[options] [-e] <local project path> ...
  D:\python\python.exe -m pip install \[options] <archive url/path> ...

no such option: -y
```

In [2]:
```
%pip install seaborn
%pip install folium
```

```
Requirement already satisfied: seaborn in D:\python\Lib\site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in D:\python\Lib\site-packages (from seaborn) (2.5.2)
Requirement already satisfied: pandas>=1.2 in D:\python\Lib\site-packages (from seaborn) (3.0.5)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in D:\python\Lib\site-packages (from seaborn) (3.11.1)
Requirement already satisfied: contourpy>=1.0.1 in D:\python\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in D:\python\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in D:\python\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in D:\python\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.5.0)
Requirement already satisfied: packaging>=20.0 in D:\python\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.3)
Requirement already satisfied: pillow>=9 in D:\python\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.3.0)
Requirement already satisfied: pyparsing>=3 in D:\python\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in D:\python\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: tzdata in D:\python\Lib\site-packages (from pandas>=1.2->seaborn) (2026.3)
Requirement already satisfied: six>=1.5 in D:\python\Lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
[notice] A new release of pip is available: 26.1.2 -> 26.2.2
[notice] To update, run: python.exe -m pip install --upgrade pip
Requirement already satisfied: folium in D:\python\Lib\site-packages (0.20.0)
Requirement already satisfied: branca>=0.6.0 in D:\python\Lib\site-packages (from folium) (0.8.2)
Requirement already satisfied: jinja2>=2.9 in D:\python\Lib\site-packages (from folium) (3.1.6)
Requirement already satisfied: numpy in D:\python\Lib\site-packages (from folium) (2.5.2)
Requirement already satisfied: requests in D:\python\Lib\site-packages (from folium) (2.34.2)
Requirement already satisfied: xyzservices in D:\python\Lib\site-packages (from folium) (2026.9.1)
Requirement already satisfied: MarkupSafe>=2.0 in D:\python\Lib\site-packages (from jinja2>=2.9->folium) (3.0.3)
Requirement already satisfied: charset_normalizer<4,>=2 in D:\python\Lib\site-packages (from requests->folium) (3.5.1)
Requirement already satisfied: idna<4,>=2.5 in D:\python\Lib\site-packages (from requests->folium) (3.19)
Requirement already satisfied: urllib3<3,>=1.26 in D:\python\Lib\site-packages (from requests->folium) (2.7.0)
Requirement already satisfied: certifi>=2023.5.7 in D:\python\Lib\site-packages (from requests->folium) (2026.7.22)
Note: you may need to restart the kernel to use updated packages.
[notice] A new release of pip is available: 26.1.2 -> 26.2.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [3]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import folium
%matplotlib inline
```

In [4]:
```
import pandas as pd

df = pd.read_csv('Historical_Wildfires.csv')
print('Data read into a pandas dataframe!')
```

```
Data read into a pandas dataframe!
```

In [5]:
```
df.head()
```

Out[5]:

	Region	Date	Estimated_fire_area	Mean_estimated_fire_brightness	Mean_estimated_fire_radiative_power	Mean_confidence	Std_confidence	Var_confidence	Count	Replaced
0	NSW	1/4/2005	8.68000	312.266667	42.400000	78.666667	2.886751	8.333333	3	R
1	NSW	1/5/2005	16.61125	322.475000	62.362500	85.500000	8.088793	65.428571	8	R
2	NSW	1/6/2005	5.52000	325.266667	38.400000	78.333333	3.214550	10.333333	3	R
3	NSW	1/7/2005	6.26400	313.870000	33.800000	92.200000	7.529940	56.700000	5	R
4	NSW	1/8/2005	5.40000	337.383333	122.533333	91.000000	7.937254	63.000000	3	R

In [6]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 26406 entries, 0 to 26405
Data columns (total 10 columns):
 #   Column                               Non-Null Count  Dtype
---  ------                               --------------  -----
 0   Region                               26406 non-null  str
 1   Date                                 26406 non-null  str
 2   Estimated_fire_area                  26406 non-null  float64
 3   Mean_estimated_fire_brightness       26406 non-null  float64
 4   Mean_estimated_fire_radiative_power  26406 non-null  float64
 5   Mean_confidence                      26406 non-null  float64
 6   Std_confidence                       24199 non-null  float64
 7   Var_confidence                       24199 non-null  float64
 8   Count                                26406 non-null  int64
 9   Replaced                             26406 non-null  str
dtypes: float64(6), int64(1), str(3)
memory usage: 2.3 MB
```

In [7]:
```
#Column names
df.columns
```

Out[7]:
```
Index(['Region', 'Date', 'Estimated_fire_area',
       'Mean_estimated_fire_brightness', 'Mean_estimated_fire_radiative_power',
       'Mean_confidence', 'Std_confidence', 'Var_confidence', 'Count',
       'Replaced'],
      dtype='str')
```

In [8]:
```
#data type
df.dtypes
```

Out[8]:
```
Region                                 str
Date                                   str
Estimated_fire_area                float64
Mean_estimated_fire_brightness     float64
Mean_estimated_fire_radiative_power    float64
Mean_confidence                    float64
Std_confidence                     float64
Var_confidence                     float64
Count                                int64
Replaced                               str
dtype: object
```

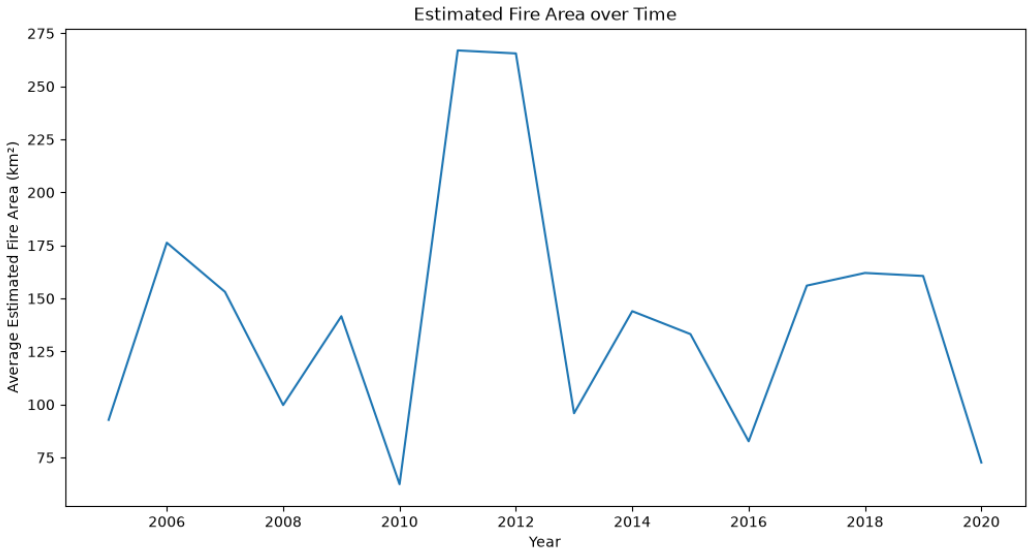
In [9]:
```
import datetime as dt

df['Year'] = pd.to_datetime(df['Date']).dt.year
df['Month'] = pd.to_datetime(df['Date']).dt.month
```

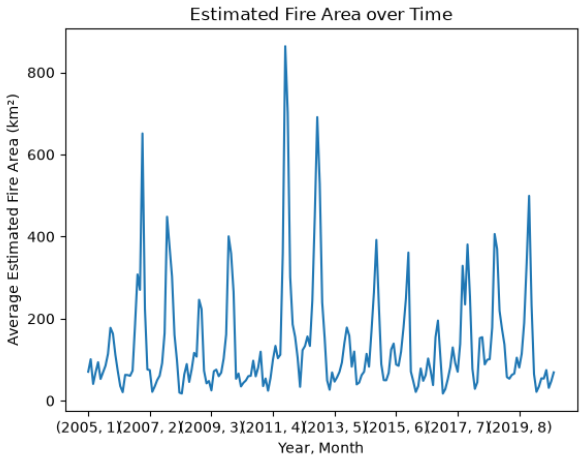
In [10]:
```
#verify the columns again
df.dtypes
```

Out[10]:
```
Region                                 str
Date                                   str
Estimated_fire_area                float64
Mean_estimated_fire_brightness     float64
Mean_estimated_fire_radiative_power    float64
Mean_confidence                    float64
Std_confidence                     float64
Var_confidence                     float64
Count                                int64
Replaced                               str
Year                               int32
Month                              int32
dtype: object
```

In [11]:
```
plt.figure(figsize=(12, 6))
# Grouping the data by 'Year' and calculating the mean of 'Estimated_fire_area'
df_new = df.groupby('Year')['Estimated_fire_area'].mean()
# Plotting the data
df_new.plot(x=df_new.index, y=df_new.values)
plt.xlabel('Year')
plt.ylabel('Average Estimated Fire Area (km²)')
plt.title('Estimated Fire Area over Time')
plt.show()
```

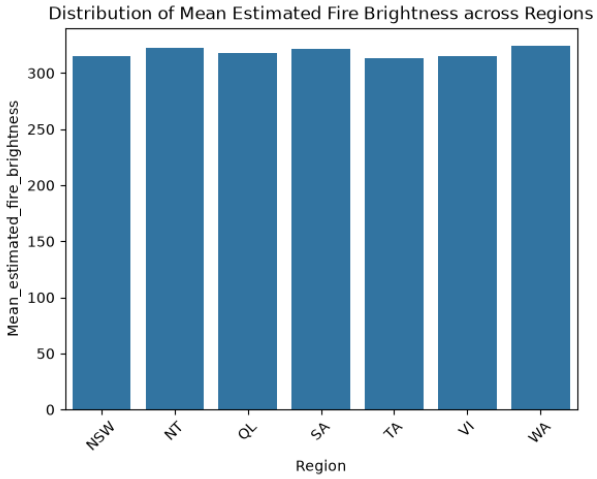


In [12]:
```
# Grouping the data by both 'Year' and 'Month', and calculating the mean of 'Estimated_fire_area'
df_new = df.groupby(['Year','Month'])['Estimated_fire_area'].mean()
# Plotting the data
df_new.plot(x=df_new.index, y=df_new.values)
plt.xlabel('Year, Month')
plt.ylabel('Average Estimated Fire Area (km²)')
plt.title('Estimated Fire Area over Time')
plt.show()
```

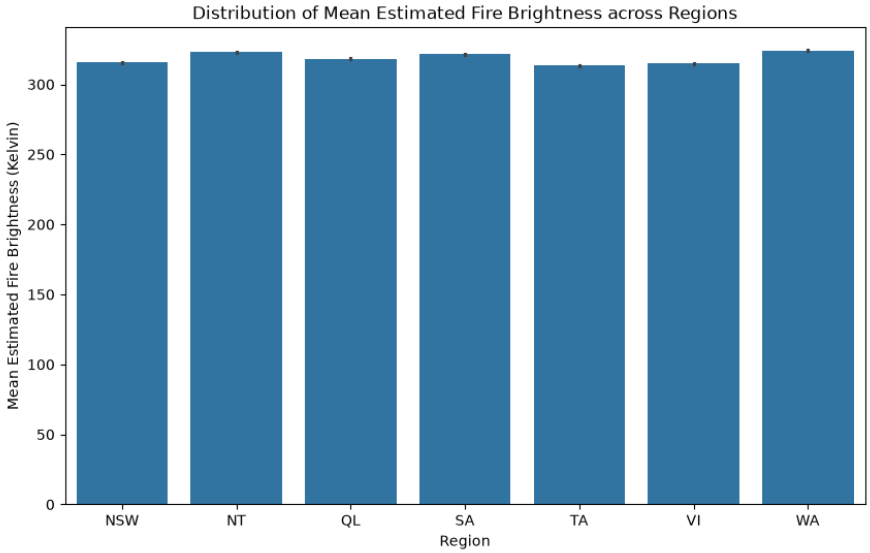


In [13]:
```
import seaborn as sns
import matplotlib.pyplot as plt

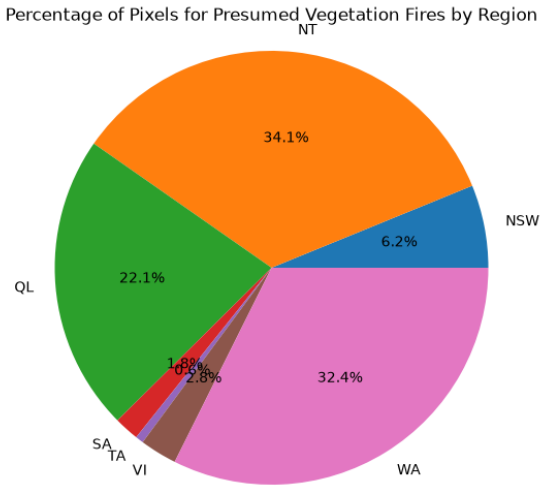
data = df.groupby("Region")["Mean_estimated_fire_brightness"].mean().reset_index()
sns.barplot(data=data, x="Region", y="Mean_estimated_fire_brightness")
plt.title("Distribution of Mean Estimated Fire Brightness across Regions")
plt.xticks(rotation=45)
plt.show()
```



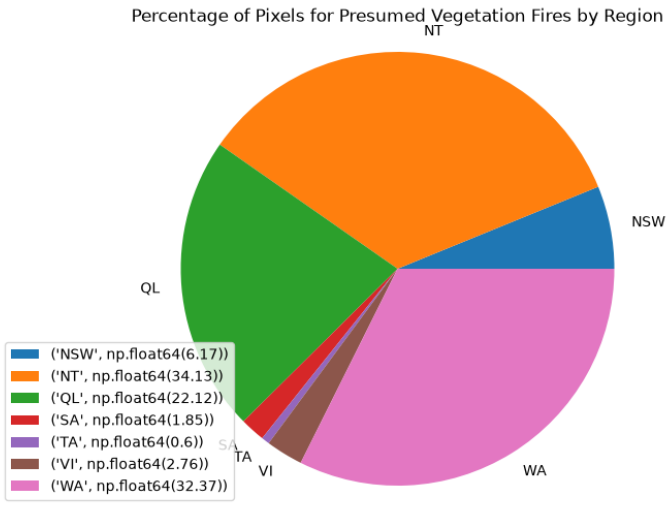
In [14]:
```
# Creating a bar plot using seaborn to visualize the distribution of mean estimated fire brightness across regions
plt.figure(figsize=(10, 6))
# Using seaborn's barplot function to create the plot
sns.barplot(data=df, x='Region', y='Mean_estimated_fire_brightness')
plt.xlabel('Region')
plt.ylabel('Mean Estimated Fire Brightness (Kelvin)')
plt.title('Distribution of Mean Estimated Fire Brightness across Regions')
plt.show()
```



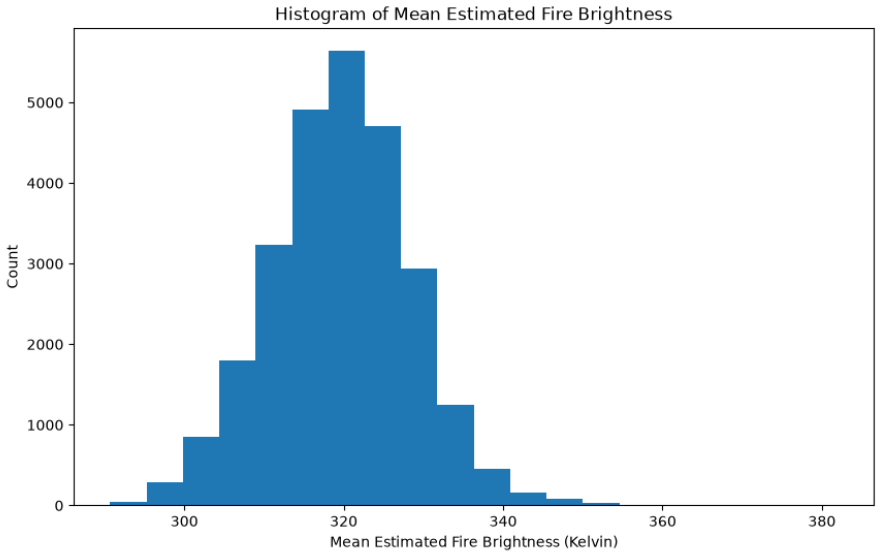
In [15]:
```
plt.figure(figsize=(10, 6))
region_counts = df.groupby('Region')['Count'].sum()
plt.pie(region_counts, labels=region_counts.index, autopct='%1.1f%%')
plt.title('Percentage of Pixels for Presumed Vegetation Fires by Region')
plt.axis('equal')
plt.show()
```



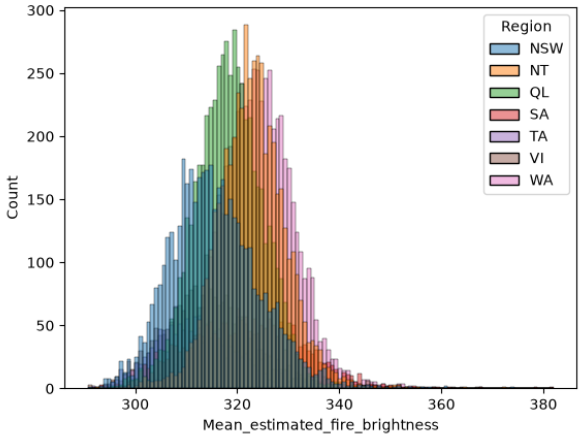
In [16]:
```
plt.figure(figsize=(10, 6))
region_counts = df.groupby('Region')['Count'].sum()
plt.pie(region_counts, labels=region_counts.index)
plt.title('Percentage of Pixels for Presumed Vegetation Fires by Region')
legend_data = [(i, round(k / region_counts.sum() * 100, 2)) for i, k in zip(region_counts.index, region_counts)]
plt.legend(legend_data)
plt.axis('equal')
plt.show()
```



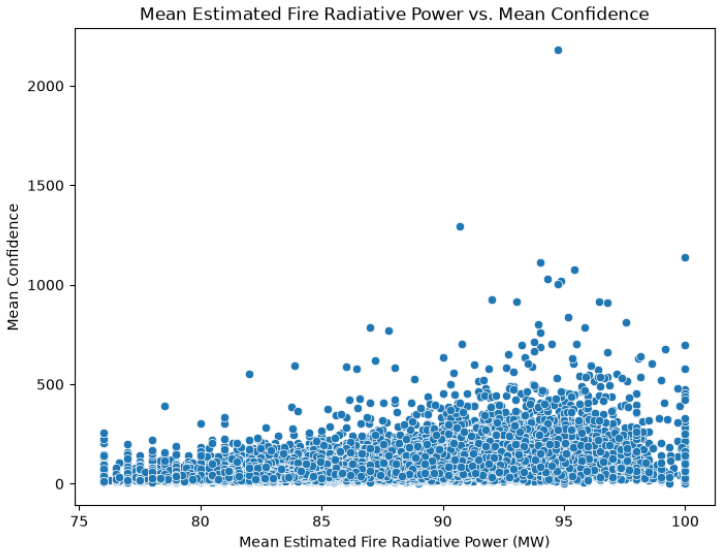
In [17]:
```
plt.figure(figsize=(10, 6))
plt.hist(x=df['Mean_estimated_fire_brightness'], bins=20)
plt.xlabel('Mean Estimated Fire Brightness (Kelvin)')
plt.ylabel('Count')
plt.title('Histogram of Mean Estimated Fire Brightness')
plt.show()
```



In [18]:
```
sns.histplot(data=df, x='Mean_estimated_fire_brightness', hue='Region')
plt.show()
```



In [19]:
```
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='Mean_confidence', y='Mean_estimated_fire_radiative_power')
plt.xlabel('Mean Estimated Fire Radiative Power (MW)')
plt.ylabel('Mean Confidence')
plt.title('Mean Estimated Fire Radiative Power vs. Mean Confidence')
plt.show()
```



In [20]:
```
region_data = {'region':['NSW','QL','SA','TA','VI','WA','NT'], 'Lat':[-31.8759835,-22.1646782,-30.5343665,-42.035067,-36.5986096,-25.2303005,-19.491411],
               'Lon':[147.2869493,144.5844903,135.6301212,146.6366887,144.6780052,121.0187246,132.550964]}
reg=pd.DataFrame(region_data)
reg
```

Out[20]:

	region	Lat	Lon
0	NSW	-31.875984	147.286949
1	QL	-22.164678	144.584490
2	SA	-30.534367	135.630121
3	TA	-42.035067	146.636689
4	VI	-36.598610	144.678005
5	WA	-25.230300	121.018725
6	NT	-19.491411	132.550964

In [21]:
```
import folium
import pandas as pd

region_data = {'region': ['NSW','QL','SA','TA','VI','WA','NT'],
               'Lat': [-32.810484, -22.485714, -30.00023, -42.032116, -37.810122, -25.042254, -19.433252],
               'Lon': [147.2869493,144.5844903,135.6301212,146.6366887,144.96298,121.4604818,132.8980726]}
reg=pd.DataFrame(region_data)
reg

aus_reg = folium.map.FeatureGroup()
Aus_map = folium.Map(location=[-25, 135], zoom_start=4)

for lat, lng, lab in zip(reg.Lat, reg.Lon, reg.region):
    aus_reg.add_child(
        folium.features.CircleMarker(
            [lat, lng],
            popup=lab,
            radius=5,
            color='red',
            fill=True,
            fill_color='blue',
            fill_opacity=0.6
        )
    )

Aus_map.add_child(aus_reg)
Aus_map
```

Out[21]:

